

Customer Churn Prediction

March 17, 2026

1 Background and Context

In highly competitive industries such as telecommunications, banking, and subscription-based digital services, customer retention is a critical business challenge. Acquiring new customers is significantly more expensive than retaining existing ones. Organizations therefore require data-driven mechanisms to identify customers who are likely to discontinue their services (churn) so that timely retention strategies can be implemented. With the availability of historical customer data, machine learning techniques can be leveraged to analyze usage patterns and behavioral indicators to predict customer churn in advance.

2 Problem Statement

The objective of this project is to develop a machine learning model that predicts whether a customer is likely to churn based on their demographic details, service usage patterns, and account-related information. The model aims to help organizations proactively identify at-risk customers and take preventive actions to improve customer retention and reduce revenue loss.

3 Business Objectives

- Reduce customer attrition by identifying high-risk customers early
- Enable targeted retention campaigns and personalized offers
- Improve overall customer lifetime value (CLV)
- Support data-driven decision-making for customer relationship management

4 Machine Learning Objectives

- Build a binary classification model to predict customer churn (Yes/No)
- Analyze key factors contributing to customer churn
- Evaluate model performance using appropriate classification metrics

5 Dataset Description

The dataset used for this project is the Telco Customer Churn Dataset, which is publicly available and widely used for machine learning classification tasks.

Dataset Link: <https://www.kaggle.com/datasets/blatchar/telco-customer-churn>

The dataset contains historical customer records with the following types of features: - Customer Demographics: Gender, senior citizen status, partner, dependents - Account Information: Tenure, contract type, payment method, billing details - Service Usage: Internet service, phone service, streaming services - Target Variable: Churn (Yes/No)

Each row represents a unique customer, and the dataset includes both numerical and categorical variables.

```
[1]: # importing libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

6 Data Loading

```
[2]: df = pd.read_csv("Customer_Churn.csv")
df.head()
```

```
[2]:  customerID  gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \
0  7590-VHVEG  Female                0      Yes           No         1           No
1  5575-GNVDE   Male                0      No            No        34           Yes
2  3668-QPYBK   Male                0      No            No         2           Yes
3  7795-CFOCW   Male                0      No            No        45           No
4  9237-HQITU   Female              0      No            No         2           Yes
```

```
      MultipleLines  InternetService  OnlineSecurity  ...  DeviceProtection  \
0  No phone service              DSL                No  ...              No
1                No              DSL                Yes  ...              Yes
2                No              DSL                Yes  ...              No
3  No phone service              DSL                Yes  ...              Yes
4                No      Fiber optic                No  ...              No
```

```
      TechSupport  StreamingTV  StreamingMovies      Contract  PaperlessBilling  \
0                No           No                No  Month-to-month              Yes
1                No           No                No      One year              No
2                No           No                No  Month-to-month              Yes
3                Yes           No                No      One year              No
4                No           No                No  Month-to-month              Yes
```

```
      PaymentMethod  MonthlyCharges  TotalCharges  Churn
0  Electronic check           29.85         29.85    No
1      Mailed check           56.95        1889.5    No
2      Mailed check           53.85         108.15   Yes
3  Bank transfer (automatic)      42.30        1840.75   No
4      Electronic check           70.70         151.65   Yes
```

[5 rows x 21 columns]

7 Exploratory Data Analysis (EDA)

In this section we explore the dataset to understand: - data types - missing values - class distribution

```
[3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   customerID            7043 non-null   object 
 1   gender                7043 non-null   object 
 2   SeniorCitizen         7043 non-null   int64  
 3   Partner               7043 non-null   object 
 4   Dependents            7043 non-null   object 
 5   tenure                7043 non-null   int64  
 6   PhoneService          7043 non-null   object 
 7   MultipleLines         7043 non-null   object 
 8   InternetService       7043 non-null   object 
 9   OnlineSecurity        7043 non-null   object 
10   OnlineBackup          7043 non-null   object 
11   DeviceProtection      7043 non-null   object 
12   TechSupport           7043 non-null   object 
13   StreamingTV           7043 non-null   object 
14   StreamingMovies       7043 non-null   object 
15   Contract              7043 non-null   object 
16   PaperlessBilling      7043 non-null   object 
17   PaymentMethod         7043 non-null   object 
18   MonthlyCharges        7043 non-null   float64 
19   TotalCharges          7043 non-null   object 
20   Churn                 7043 non-null   object 
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
[4]: df.isnull().sum()
```

```
[4]: customerID      0
     gender         0
     SeniorCitizen  0
     Partner        0
     Dependents     0
     tenure         0
     PhoneService   0
     MultipleLines  0
```

```

InternetService      0
OnlineSecurity       0
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
MonthlyCharges       0
TotalCharges         0
Churn                0
dtype: int64

```

There is a critical observation here. In the column 'TotalCharges', we see 'object' datatype but this column is primarily float type. Also, the null count is zero for the same. Possible reasons may be some entries as blank spaces or some string data stored there.

```
[5]: df[pd.to_numeric(df['TotalCharges'], errors='coerce').isnull()]
```

```
[5]:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	\
488	4472-LVYGI	Female	0	Yes	Yes	0	
753	3115-CZMZD	Male	0	No	Yes	0	
936	5709-LVOEQ	Female	0	Yes	Yes	0	
1082	4367-NUYAO	Male	0	Yes	Yes	0	
1340	1371-DWPAZ	Female	0	Yes	Yes	0	
3331	7644-OMVMY	Male	0	Yes	Yes	0	
3826	3213-VVOLG	Male	0	Yes	Yes	0	
4380	2520-SGTTA	Female	0	Yes	Yes	0	
5218	2923-ARZLG	Male	0	Yes	Yes	0	
6670	4075-WKNIU	Female	0	Yes	Yes	0	
6754	2775-SEFEE	Male	0	No	Yes	0	

	PhoneService	MultipleLines	InternetService	OnlineSecurity	...	\
488	No	No phone service	DSL	Yes	...	
753	Yes	No	No	No internet service	...	
936	Yes	No	DSL	Yes	...	
1082	Yes	Yes	No	No internet service	...	
1340	No	No phone service	DSL	Yes	...	
3331	Yes	No	No	No internet service	...	
3826	Yes	Yes	No	No internet service	...	
4380	Yes	No	No	No internet service	...	
5218	Yes	No	No	No internet service	...	
6670	Yes	Yes	DSL	No	...	
6754	Yes	Yes	DSL	Yes	...	

```

DeviceProtection      TechSupport      StreamingTV  \

```

488	Yes	Yes	Yes
753	No internet service	No internet service	No internet service
936	Yes	No	Yes
1082	No internet service	No internet service	No internet service
1340	Yes	Yes	Yes
3331	No internet service	No internet service	No internet service
3826	No internet service	No internet service	No internet service
4380	No internet service	No internet service	No internet service
5218	No internet service	No internet service	No internet service
6670	Yes	Yes	Yes
6754	No	Yes	No

	StreamingMovies	Contract	PaperlessBilling	\
488	No	Two year	Yes	
753	No internet service	Two year	No	
936	Yes	Two year	No	
1082	No internet service	Two year	No	
1340	No	Two year	No	
3331	No internet service	Two year	No	
3826	No internet service	Two year	No	
4380	No internet service	Two year	No	
5218	No internet service	One year	Yes	
6670	No	Two year	No	
6754	No	Two year	Yes	

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
488	Bank transfer (automatic)	52.55		No
753	Mailed check	20.25		No
936	Mailed check	80.85		No
1082	Mailed check	25.75		No
1340	Credit card (automatic)	56.05		No
3331	Mailed check	19.85		No
3826	Mailed check	25.35		No
4380	Mailed check	20.00		No
5218	Mailed check	19.70		No
6670	Mailed check	73.35		No
6754	Bank transfer (automatic)	61.90		No

[11 rows x 21 columns]

Above are the rows where TotalCharges has non numeric (blank space) values. We will convert them to NaN

```
[6]: df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
```

```
[7]: df.isnull().sum()
```

```

[7]: customerID      0
    gender           0
    SeniorCitizen    0
    Partner          0
    Dependents       0
    tenure           0
    PhoneService     0
    MultipleLines    0
    InternetService  0
    OnlineSecurity   0
    OnlineBackup     0
    DeviceProtection 0
    TechSupport      0
    StreamingTV      0
    StreamingMovies  0
    Contract         0
    PaperlessBilling 0
    PaymentMethod    0
    MonthlyCharges   0
    TotalCharges     11
    Churn            0
    dtype: int64

```

Now we clearly see there are 11 missing values in the TotalCharges column. Let's drop those rows with missing values.

```

[8]: df=df.dropna()

```

```

[9]: df.isnull().sum()

```

```

[9]: customerID      0
    gender           0
    SeniorCitizen    0
    Partner          0
    Dependents       0
    tenure           0
    PhoneService     0
    MultipleLines    0
    InternetService  0
    OnlineSecurity   0
    OnlineBackup     0
    DeviceProtection 0
    TechSupport      0
    StreamingTV      0
    StreamingMovies  0
    Contract         0
    PaperlessBilling 0
    PaymentMethod    0

```

```
MonthlyCharges      0
TotalCharges         0
Churn                0
dtype: int64
```

Now we have handled all null values. But some columns need re-adjustment in their values.

Converting 'No internet service' to 'No'.

```
[10]: cols = [
      'OnlineSecurity',
      'OnlineBackup',
      'DeviceProtection',
      'TechSupport',
      'StreamingTV',
      'StreamingMovies'
    ]

    for col in cols:
        df[col] = df[col].replace('No internet service', 'No')
```

Converting 'No phone service' to 'No'.

```
[11]: df['MultipleLines'] = df['MultipleLines'].replace('No phone service', 'No')
```

Also drop customerID column because neither does it provide any insight nor help with ML models.

```
[12]: df = df.drop('customerID', axis = 1)
```

Now let's take a glance of statistical data.

```
[13]: df.describe()
```

```
[13]:
```

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7032.000000	7032.000000	7032.000000	7032.000000
mean	0.162400	32.421786	64.798208	2283.300441
std	0.368844	24.545260	30.085974	2266.771362
min	0.000000	1.000000	18.250000	18.800000
25%	0.000000	9.000000	35.587500	401.450000
50%	0.000000	29.000000	70.350000	1397.475000
75%	0.000000	55.000000	89.862500	3794.737500
max	1.000000	72.000000	118.750000	8684.800000

7.1 EDA - Visualization

In this section, we explore the dataset to understand the relationship between different features and customer churn. The goal is to identify patterns and potential drivers of churn that may help improve our machine learning model.

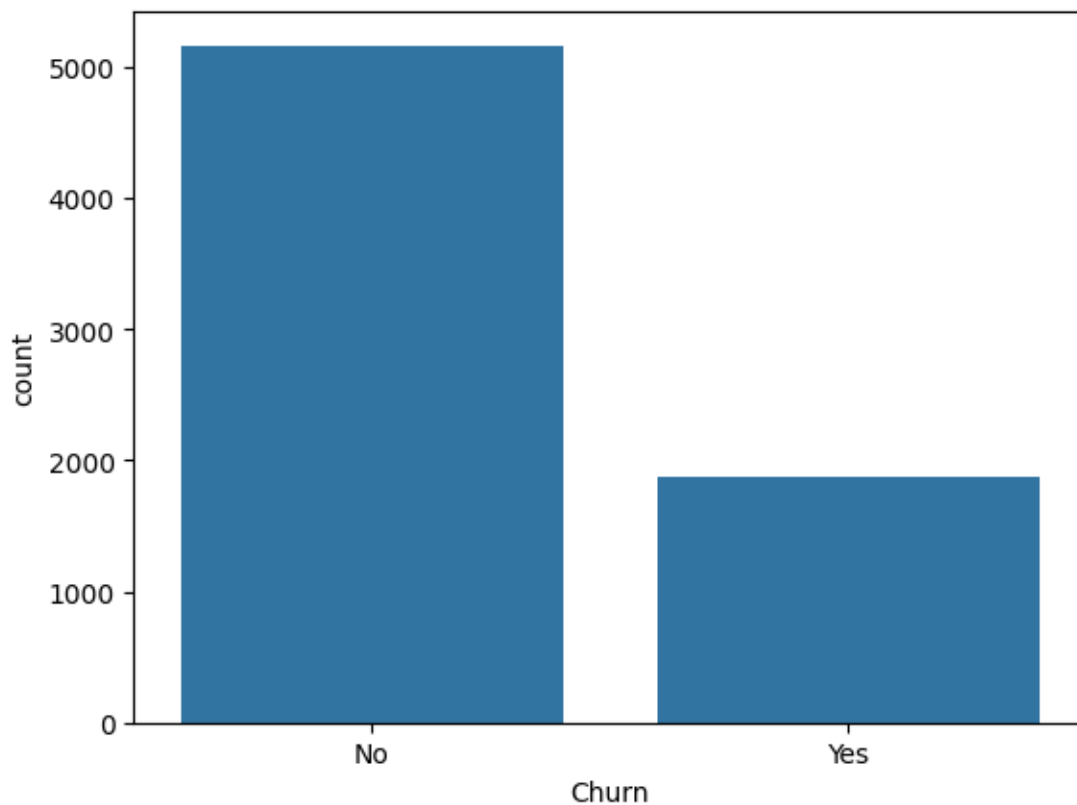
7.1.1 Plots used:

1. Churn Distribution
2. Contract Type vs Churn
3. Tenure vs Churn
4. Monthly Charges vs Churn
5. Total Charges vs Churn
6. Internet Service vs Churn
7. Payment Method vs Churn
8. Correlation Heatmap

1. Churn Distribution

```
[14]: sns.countplot(x='Churn', data=df)
```

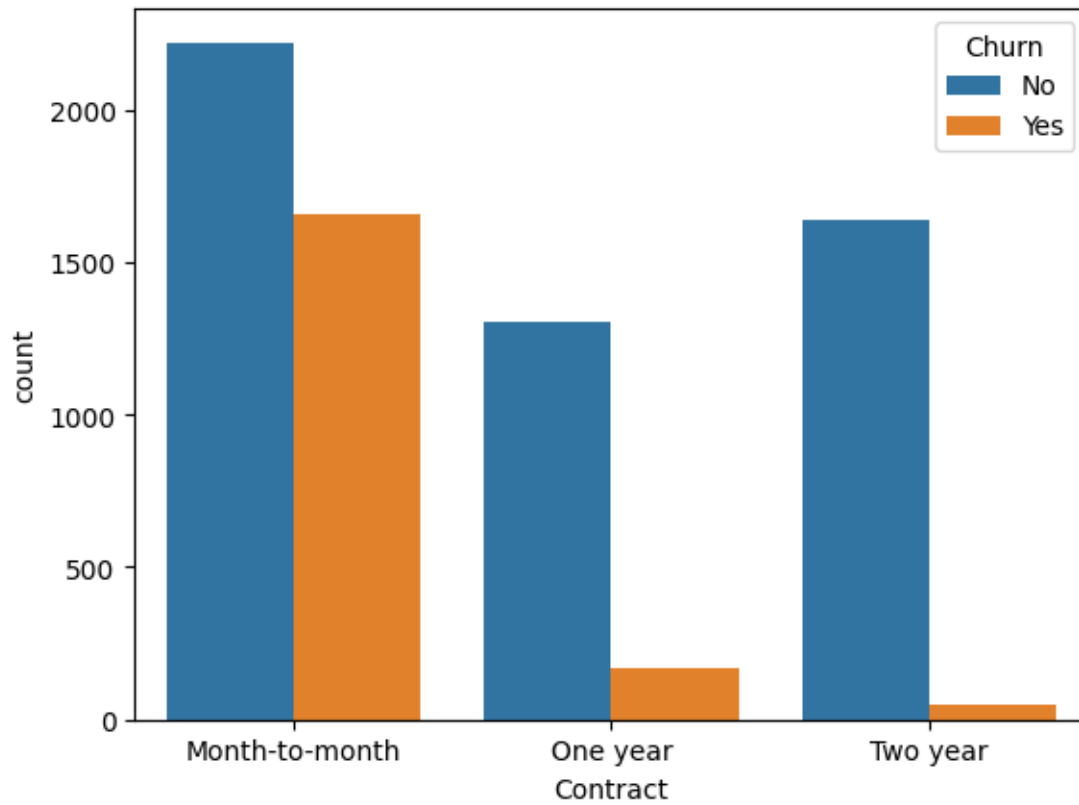
```
[14]: <Axes: xlabel='Churn', ylabel='count'>
```



2. Contract Type vs Customer Churn

```
[15]: sns.countplot(x='Contract', hue='Churn', data=df)
```

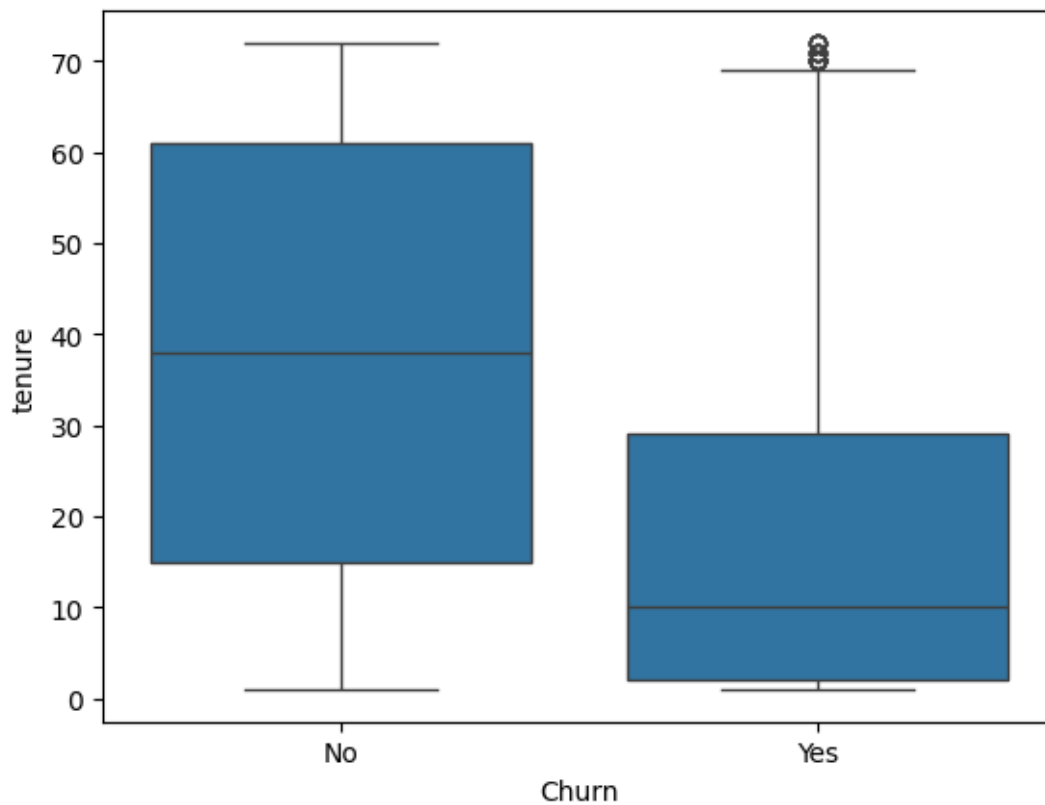
```
[15]: <Axes: xlabel='Contract', ylabel='count'>
```



3. Tenure vs Churn

```
[16]: sns.boxplot(x='Churn', y='tenure', data=df)
```

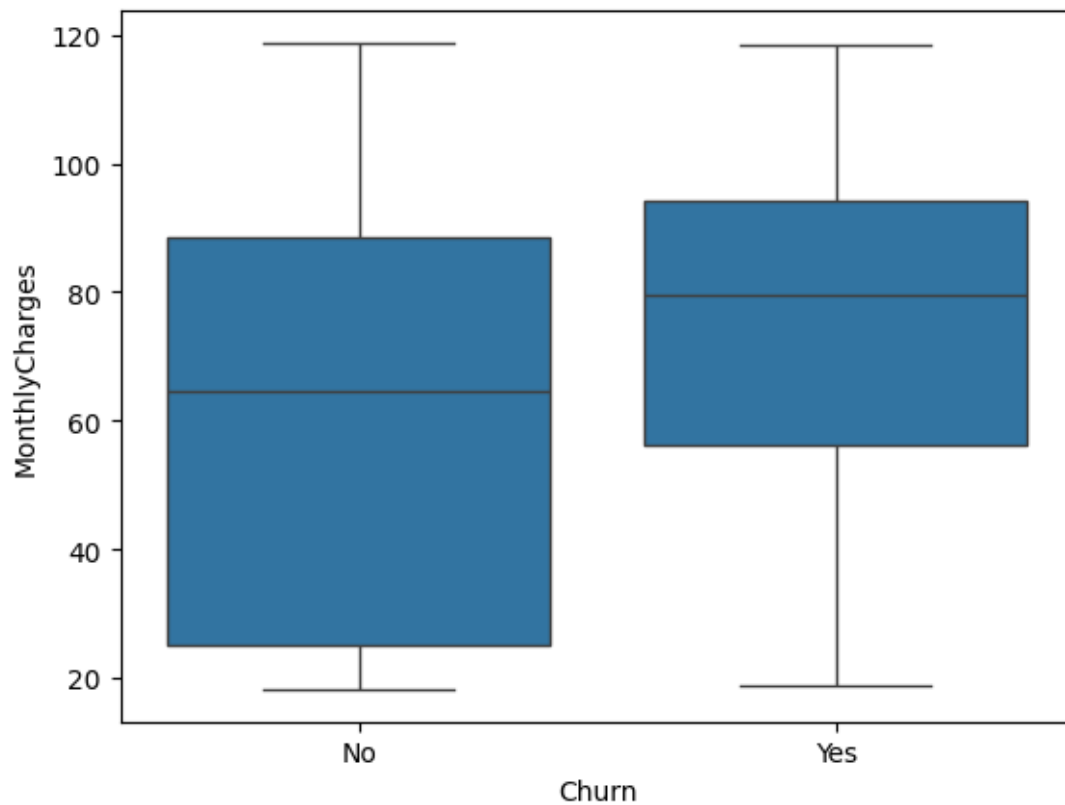
```
[16]: <Axes: xlabel='Churn', ylabel='tenure'>
```



4. Monthly Charges vs Churn

```
[17]: sns.boxplot(x='Churn', y='MonthlyCharges', data=df)
```

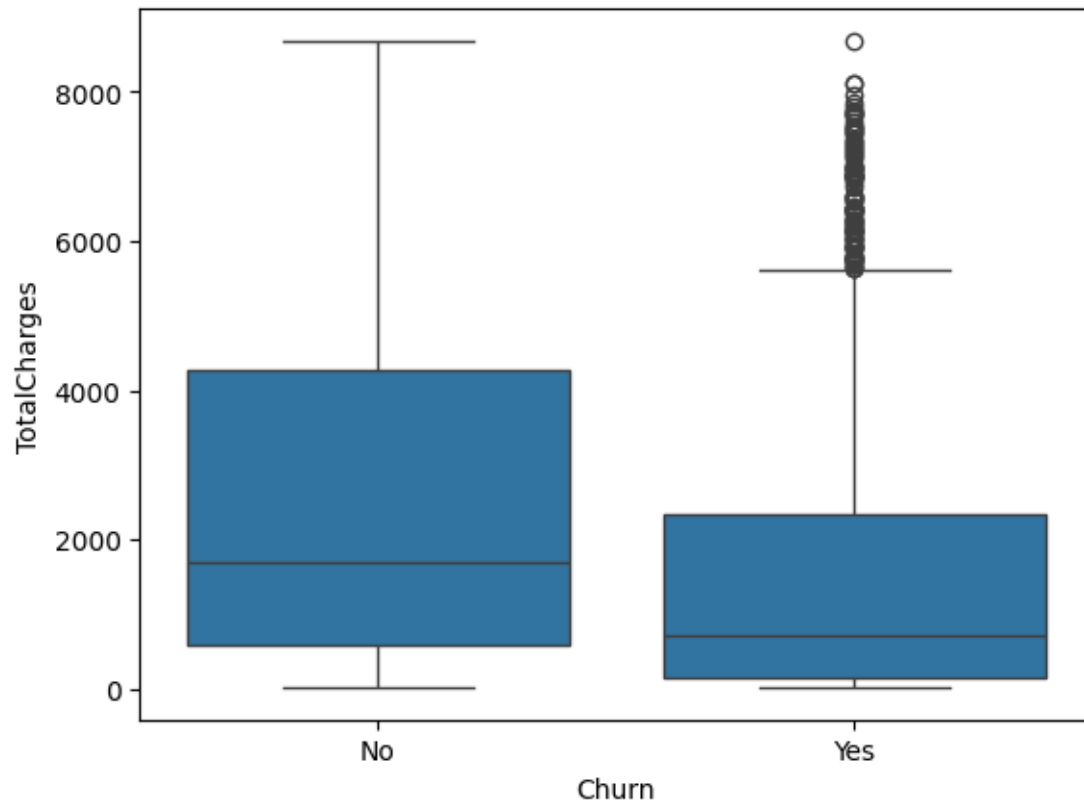
```
[17]: <Axes: xlabel='Churn', ylabel='MonthlyCharges'>
```



5. Total Charges vs Churn

```
[18]: sns.boxplot(x='Churn', y='TotalCharges', data=df)
```

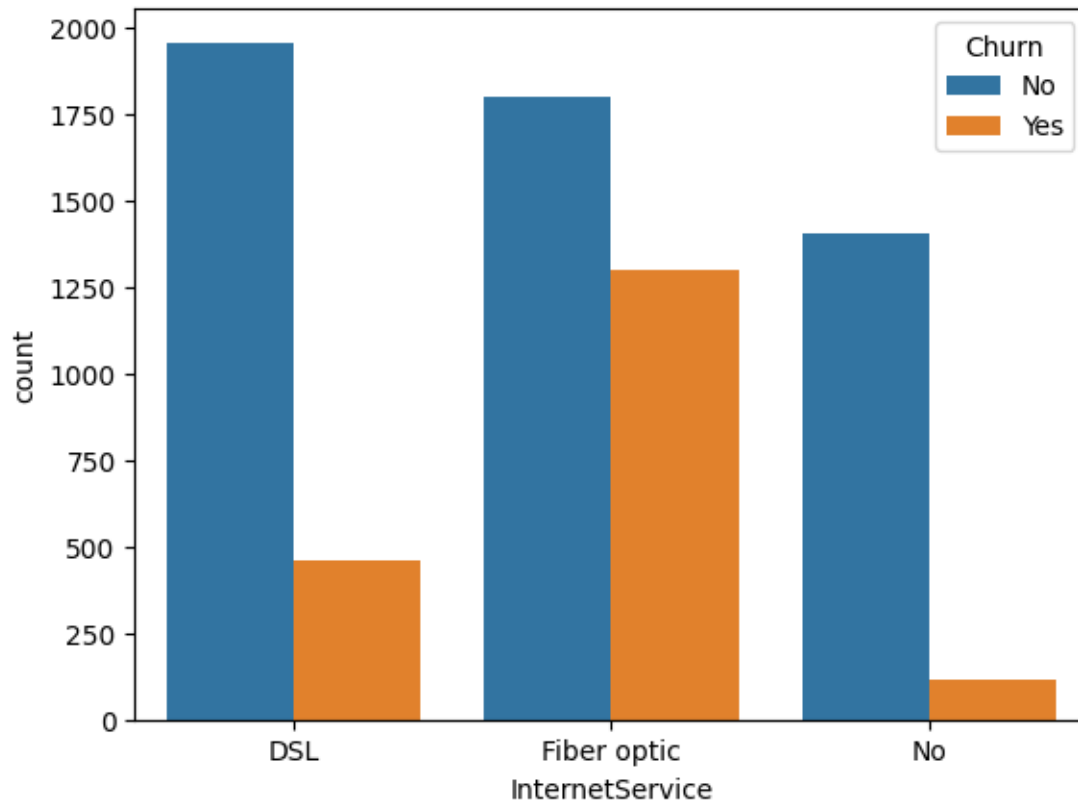
```
[18]: <Axes: xlabel='Churn', ylabel='TotalCharges'>
```



6. Internet Service vs Churn

```
[19]: sns.countplot(x='InternetService', hue='Churn', data=df)
```

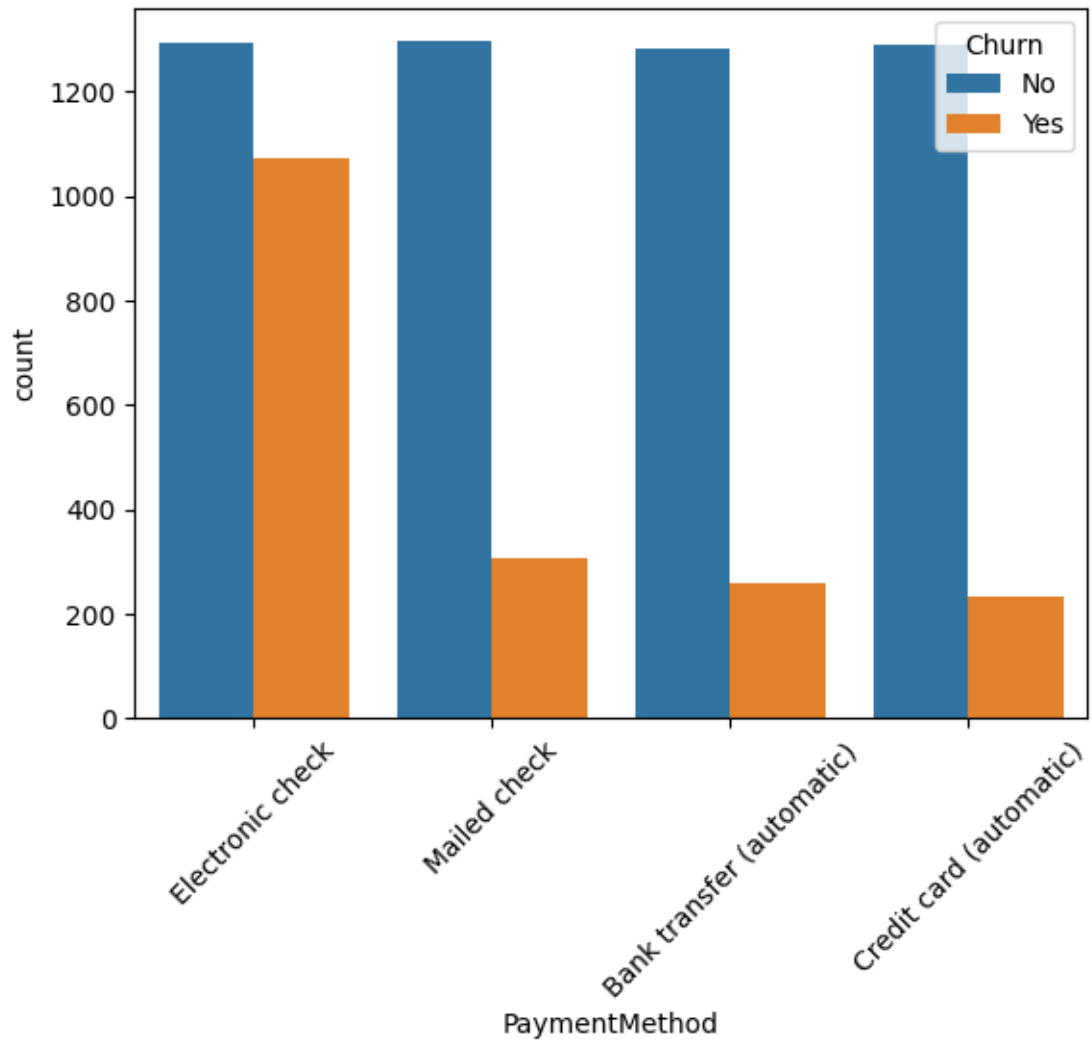
```
[19]: <Axes: xlabel='InternetService', ylabel='count'>
```



7. Payment Method vs Churn

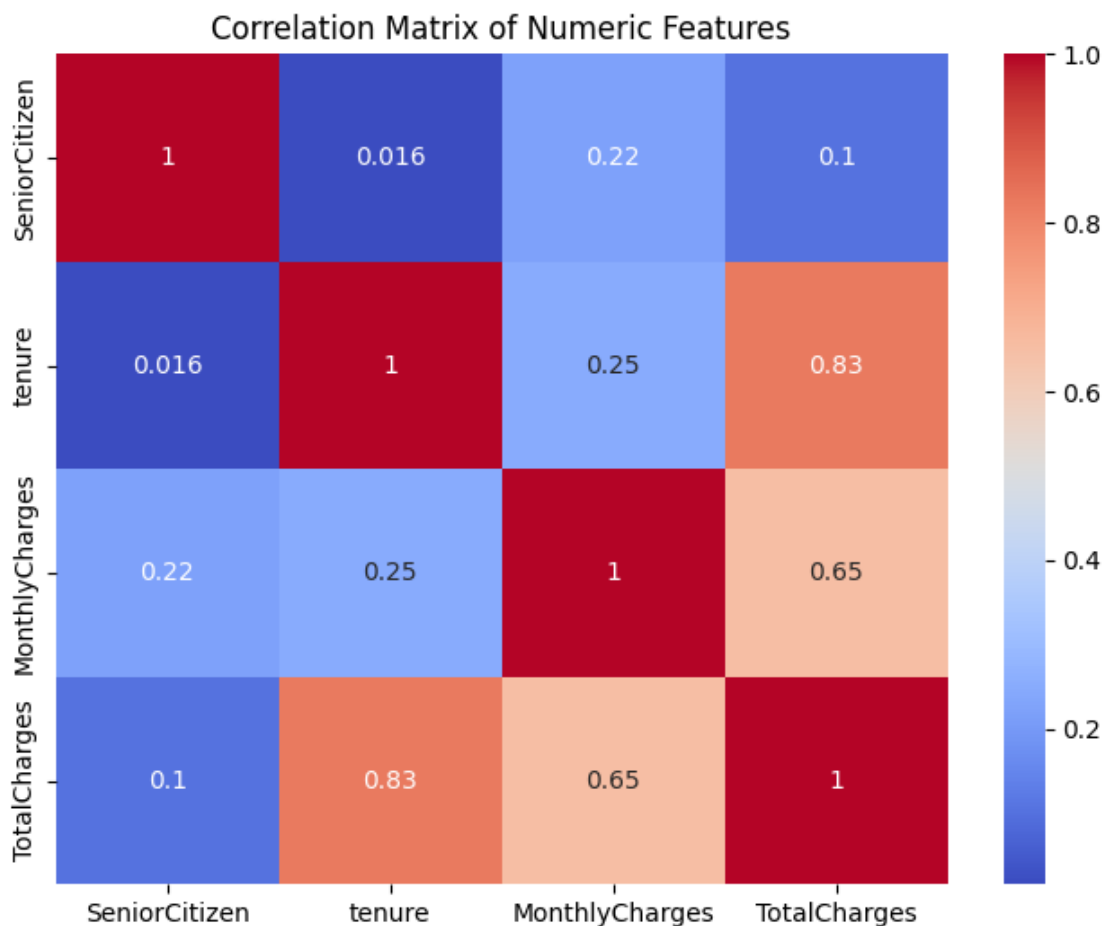
```
[20]: sns.countplot(x='PaymentMethod', hue='Churn', data=df)
      plt.xticks(rotation=45)
```

```
[20]: ([0, 1, 2, 3],
      [Text(0, 0, 'Electronic check'),
       Text(1, 0, 'Mailed check'),
       Text(2, 0, 'Bank transfer (automatic)'),
       Text(3, 0, 'Credit card (automatic)')])
```



8. Correlation Heatmap

```
[21]: plt.figure(figsize=(8,6))
sns.heatmap(df.select_dtypes(include='number').corr(), annot=True,
            cmap='coolwarm')
plt.title("Correlation Matrix of Numeric Features")
plt.show()
```



7.1.2 Key Insights from EDA - Visualization

1. Customers with month-to-month contracts exhibit the highest churn.
2. Customers with lower tenure are more likely to churn.
3. Higher monthly charges correlate with increased churn risk.
4. Fiber optic users show higher churn compared to DSL users.
5. Electronic check payment method is associated with higher churn.
6. The customers with lower total charges are likely to churn.

These insights suggest that pricing structure, contract commitment, and service type significantly influence churn behavior.

8 Machine Learning

In this section, we build machine learning models to predict customer churn.

The workflow includes preprocessing using pipelines, hyperparameter tuning using GridSearchCV, and model comparison based on ROC-AUC and classification metrics.

8.1 Train-Test Split

```
[22]: X = df.drop('Churn', axis=1)
df['Churn'] = df['Churn'].map({'No':0, 'Yes':1})
y = df['Churn']
```

```
[23]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
↪random_state = 54)
```

8.2 Defining Column Types

```
[24]: numeric_cols = X.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = X.select_dtypes(include='object').columns
```

8.3 Preprocessing Transformer

```
[25]: from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer(
    transformers = [
        ('num', StandardScaler(), numeric_cols),
        ('cat', OneHotEncoder(), categorical_cols)
    ]
)
```

8.4 Importing Models

```
[26]: from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

8.5 Creating Model Dictionary

```
[27]: models = {
    "Logistic Regression" : {
        'model' : LogisticRegression(max_iter=1000),
        'params' : {
            "model__C" : [0.01, 0.1, 1, 10]
        }
    },
    "Decision Tree" : {
        'model' : DecisionTreeClassifier(),
        'params' : {
            "model__max_depth" : [3,5,7,10],
            "model__min_samples_split" : [2,5,10]
        }
    }
}
```

```

    }
},
"Random Forest" : {
    'model' : RandomForestClassifier(),
    'params' : {
        "model__n_estimators" : [100,200],
        "model__max_depth" : [None, 10, 20]
    }
}
}

```

Note : The `class_weight` parameter of `LogisticRegression()` was tested but ultimately removed, as it slightly reduced ROC-AUC performance. Instead, class imbalance was handled through ROC-based threshold optimization, which allows the model to adjust decision boundaries without altering probability calibration.

8.6 Importing pipeline and GridSearch CV

```
[28]: from sklearn.pipeline import Pipeline
      from sklearn.model_selection import GridSearchCV
      from sklearn.model_selection import StratifiedKFold

```

8.7 Define Cross Validation

```
[29]: cv = StratifiedKFold(n_splits = 5, shuffle = True, random_state = 54)

```

8.8 Import all Metrics

```
[30]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

```

8.9 Loop Through Models

```
[31]: results = []
      best_models = {}

      for model, config in models.items():

          pipeline = Pipeline([
              ('Preprocessing', preprocessor),
              ('model', config['model'])
          ])

          grid = GridSearchCV(
              pipeline,
              config['params'],
              cv=cv,

```

```

        scoring='roc_auc',
        n_jobs=-1
    )

    grid.fit(X_train, y_train)

    best_model = grid.best_estimator_

    # store the trained model
    best_models[model] = best_model

    y_prob = best_model.predict_proba(X_test)[:,-1]

    from sklearn.metrics import roc_curve
    fpr, tpr, thresholds = roc_curve(y_test, y_prob)

    optimal_idx = (tpr - fpr).argmax()
    optimal_threshold = thresholds[optimal_idx]

    y_pred = (y_prob >= optimal_threshold).astype(int)

    results.append({
        'Model': model,
        'Optimal_Threshold': optimal_threshold,
        'Accuracy_Score': accuracy_score(y_test, y_pred),
        'Precision_Score': precision_score(y_test, y_pred),
        'Recall_Score': recall_score(y_test, y_pred),
        'F1_Score': f1_score(y_test, y_pred),
        'ROC_AUC_Score': roc_auc_score(y_test, y_prob)
    })

results_df = pd.DataFrame(results)
results_df = results_df.sort_values('ROC_AUC_Score', ascending=False)

best_model_name = results_df.iloc[0]['Model']
best_model = best_models[best_model_name]

```

8.10 Model Comparison

```

[32]: results_df = results_df.sort_values('ROC_AUC_Score', ascending=False)
      results_df = results_df.round(3)
      results_df

```

```

[32]:
      Model  Optimal_Threshold  Accuracy_Score  Precision_Score  \
0  Logistic Regression        0.310          0.778           0.566

```

2	Random Forest	0.267	0.760	0.538
1	Decision Tree	0.252	0.737	0.509

	Recall_Score	F1_Score	ROC_AUC_Score
0	0.764	0.650	0.848
2	0.798	0.643	0.841
1	0.780	0.616	0.828

8.11 Threshold Optimization for Churn Prediction

In classification models, predictions are typically converted from probabilities to class labels using a default threshold of 0.5. However, in churn prediction problems this threshold may not be optimal. Instead of using a fixed threshold, the optimal threshold was determined using the ROC curve. Specifically, the threshold that maximizes Youden's J statistic was selected:

$$J = \text{TPR} - \text{FPR}$$

Where:

TPR (True Positive Rate) represents the proportion of churners correctly identified.

FPR (False Positive Rate) represents the proportion of non-churners incorrectly flagged.

Maximizing TPR–FPR identifies the point where the model best separates churners from non-churners.

In practical business applications, this threshold may be further adjusted depending on the cost of customer churn versus the cost of retention incentives.

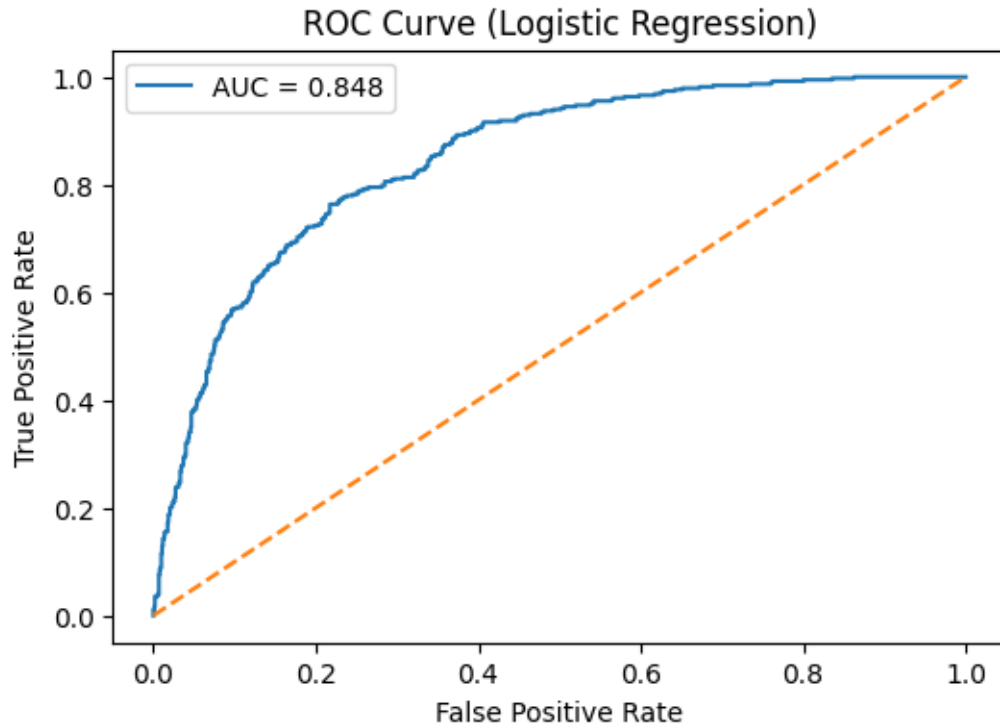
8.12 ROC Curve

```
[33]: from sklearn.metrics import roc_curve, auc

y_prob = best_model.predict_proba(X_test)[:,-1]

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
plt.plot([0,1],[0,1], '--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title(f"ROC Curve ({best_model_name})")
plt.legend()
plt.show()
```



9 Feature Importance Analysis

```
[34]: from sklearn.inspection import permutation_importance

perm_importance = permutation_importance(
    best_model,
    X_test,
    y_test,
    n_repeats=10,
    random_state=54,
    n_jobs=-1
)

importance_df = pd.DataFrame({
    "Feature": X_test.columns,
    "Importance": perm_importance.importances_mean
})

importance_df = importance_df.sort_values(by="Importance", ascending=False)

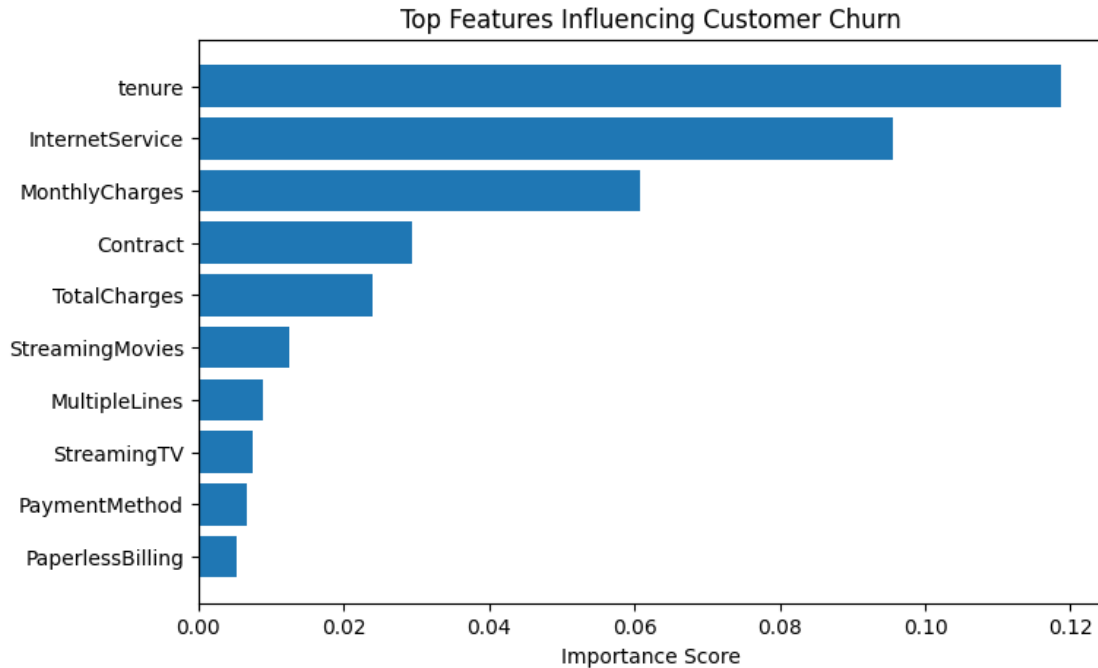
importance_df.head(110)
```

```
[34]:
```

	Feature	Importance
4	tenure	1.187633e-01
7	InternetService	9.552239e-02
17	MonthlyCharges	6.069652e-02
14	Contract	2.949538e-02
18	TotalCharges	2.395167e-02
13	StreamingMovies	1.257996e-02
6	MultipleLines	8.813077e-03
12	StreamingTV	7.533760e-03
16	PaymentMethod	6.751955e-03
15	PaperlessBilling	5.330490e-03
11	TechSupport	2.700782e-03
8	OnlineSecurity	2.558635e-03
3	Dependents	1.492537e-03
1	SeniorCitizen	1.421464e-03
9	OnlineBackup	4.975124e-04
10	DeviceProtection	4.975124e-04
2	Partner	2.132196e-04
0	gender	0.000000e+00
5	PhoneService	-3.330669e-17

```
[35]: top_features = importance_df.head(10)

plt.figure(figsize=(8,5))
plt.barh(top_features["Feature"], top_features["Importance"])
plt.gca().invert_yaxis()
plt.title("Top Features Influencing Customer Churn")
plt.xlabel("Importance Score")
plt.show()
```



10 Actionable Insights for Business Stakeholders

Based on the model analysis and feature importance results, several key factors influencing customer churn were identified. These insights can guide targeted retention strategies.

10.1 1. Focus on Customers with Low Tenure

Customers with **short tenure** show a significantly higher probability of churn. This indicates that the initial months of the customer lifecycle are critical.

Recommended actions: - Provide onboarding support and tutorials. - Offer early loyalty incentives for customers within the first 6–12 months. - Create welcome discounts or bundled service offers.

10.2 2. Address Risks in Month-to-Month Contracts

Customers on **month-to-month contracts** are more likely to churn compared to those with longer-term agreements.

Recommended actions: - Offer discounts for switching to **annual or multi-year contracts**. - Provide loyalty rewards for long-term commitments. - Introduce flexible upgrade plans to reduce switching behavior.

10.3 3. Analyze Pricing Sensitivity in Early-Stage Customers

Monthly charges alone are **not** a reliable indicator of churn. However, when combined with tenure, a clear pattern emerges: customers with high monthly charges and low tenure are more likely to churn, while long-term customers with similar charges tend to remain loyal.

This suggests that **pricing sensitivity is highest during the early stages** of the customer lifecycle.

Recommended actions:

- Offer discounted or promotional pricing for new customers in their initial months.
 - Avoid exposing new users to high-cost plans without demonstrating value.
 - Introduce gradual pricing upgrades after customer engagement stabilizes.
 - Monitor high-paying new customers as a high-risk churn segment.
-

10.4 4. Promote Value-Added Services

Services such as **Online Security, Tech Support, and Device Protection** often correlate with lower churn.

Recommended actions: - Bundle these services with standard plans. - Provide limited-time free trials to encourage adoption. - Highlight these services during customer onboarding.

10.5 5. Implement Proactive Retention Campaigns

The churn prediction model can identify **high-risk customers in advance**, enabling proactive retention strategies.

Recommended actions: - Trigger retention offers when predicted churn probability exceeds the optimized threshold. - Use personalized communication and targeted promotions. - Assign customer success teams to high-risk accounts.

11 Strategic Impact

By combining predictive modeling with targeted interventions, telecom companies can:

- Reduce customer churn
- Increase customer lifetime value
- Optimize marketing and retention spending

These insights enable a **data-driven approach to customer retention**, improving both profitability and customer satisfaction.

12 Conclusion

This project developed a machine learning framework to predict customer churn in a telecom dataset. After performing data preprocessing, exploratory data analysis, and feature engineering, multiple models were trained and evaluated using cross-validation and ROC-AUC scoring.

Among the models tested, **Logistic Regression achieved the highest ROC-AUC score**, indicating strong ability to distinguish between customers who churn and those who remain.

To improve classification performance, the default probability threshold of **0.5** was replaced with **an optimized threshold derived from the ROC curve**, improving the balance between recall and false positive rates.

Feature importance analysis highlighted **tenure, contract type, and monthly charges** as key drivers of churn, providing actionable insights for business stakeholders.

Overall, this project demonstrates a complete end-to-end machine learning workflow — from data exploration to predictive modeling and business interpretation — enabling telecom companies to proactively identify high-risk customers and implement targeted retention strategies.

12.1 Deployment

```
[36]: # retrain on full dataset
      # Save the already-trained best_model, do NOT retrain it
      import joblib

      # Save model AND optimal threshold together
      optimal_threshold = results_df.iloc[0]['Optimal_Threshold']

      joblib.dump({
          'model': best_model,
          'threshold': optimal_threshold
      }, "churn_model.pkl")
```

```
[36]: ['churn_model.pkl']
```